

UNIVERSITÉ DES ANTILLES

Campus de Schoelcher — Martinique
Licence 3 Informatique — Année 2025-2026

RAPPORT DE PROJET

FirstApp

Application Bancaire Mobile

Flutter · Laravel · Stripe · Algorand · Resend

Projet réalisé dans le cadre de la formation L3 Informatique

ELOIDIN—MICHANOL Mathis

Table des matières

Table des matières	2
1. Introduction	5
1.1 Contexte pédagogique	5
1.2 Technologies utilisées	5
2. Arborescence du projet	6
2.1 Structure générale	6
2.2 Application Flutter (firstapp/)	6
2.3 Backend Laravel (mon_api_flutter/)	6
2.4 Base de données — Migrations	6
3. Fonctionnalités développées	8
3.1 Authentification & Gestion de session	8
3.1.1 Inscription	8
3.1.2 Connexion & Auto-login	8
3.2 Portefeuille & Affichage du solde	8
3.3 Virements sécurisés par PIN	8
3.3.1 Flux complet du virement	8
3.3.2 Sécurité du virement	9
3.4 Paiement par QR Code	9
3.4.1 Génération du QR Code	9
3.4.2 Scanner & Paiement	9
3.5 Recharge via Stripe	9
3.5.1 Flux de paiement Stripe	9
3.5.2 Protection anti-doublon	10
3.6 Notifications par e-mail	10
3.7 Historique des transactions	10
3.8 Blockchain Algorand — Cryptomonnaie BKN	10
3.9 Chatbot IA	11
4. Sécurisation de l'application	12
4.1 Authentification et tokens	12
4.2 Protection des virements par PIN	12
4.3 Hachage des mots de passe	12
4.4 Validation CSRF et Webhook	12
4.5 Anti-doublon sur les transactions	12
4.6 Transactions atomiques	12

4.7 Séparation des routes publiques et protégées.....	12
4.8 Persistance locale sécurisée	12
5. Explication des API utilisées.....	14
5.1 Laravel Sanctum.....	14
5.2 Stripe.....	14
5.3 Resend.....	14
5.4 Algorand.....	14
5.5 APIs Flutter (packages).....	14
6. Difficultés rencontrées et solutions apportées	16
6.1 Problème CORS et accès réseau local.....	16
6.2 Webhooks Stripe en développement local.....	16
6.3 Compatibilité Stripe sur Android	16
6.4 Gestion des doublons de transactions.....	16
6.5 Synchronisation du solde après virement	16
6.6 Intégration d'un Chatbot et du BKN (crypto) :	16
7. Ce qu'il manque pour une publication sur les Stores	17
7.1 Prérequis communs iOS & Android	17
7.2 App Store (iOS).....	17
7.3 Google Play Store (Android)	17
7.4 Conformité réglementaire	17
8. Architecture Frontend & Backend	19
8.1 Frontend Flutter.....	19
8.1.1 Pattern Singleton pour l'ApiService	19
8.1.2 Gestion des états	19
8.1.3 Navigation	19
8.2 Backend Laravel	19
8.2.1 Architecture MVC.....	19
8.2.2 Relations Eloquent.....	19
8.2.3 Système de notification par e-mail	19
9. Scalabilité & Perspectives d'évolution.....	20
9.1 Améliorations techniques immédiates	20
9.2 Nouvelles fonctionnalités	20
9.3 Infrastructure de production	20
10. Conclusion.....	21
Annexes.....	22

Annexe A — Routes API complètes	22
Annexe B — Variables d'environnement (.env)	22
Annexe C — Dépendances Flutter (pubspec.yaml)	22

1. Introduction

Ce rapport présente le projet FirstApp, une application bancaire mobile développée dans le cadre de la Licence 3 Informatique à l'Université des Antilles, Campus de Schoelcher (Martinique). Ce projet a été réalisé en équipe durant l'année universitaire 2025-2026.

L'objectif principal était de concevoir une application mobile complète de type fintech, permettant à des utilisateurs de gérer un portefeuille numérique, d'effectuer des virements sécurisés, de payer via QR code, de recharger leur compte par carte bancaire, d'échanger de la cryptomonnaie sur la blockchain Algorand, et d'interagir avec un chatbot d'assistance.

Le développement a été organisé via GitHub, chaque étudiant travaillant sur son application sur une branche dédiée. Cette approche a permis d'expérimenter les bonnes pratiques du travail dans un environnement professionnel.

1.1 Contexte pédagogique

L'application FirstApp s'inscrit dans une démarche d'apprentissage par la pratique. Les étudiants ont eu l'opportunité de travailler avec des technologies réelles utilisées dans le secteur fintech : API de paiement Stripe, blockchain Algorand, envoi d'e-mails transactionnels, et développement mobile multi-plateforme avec Flutter.

1.2 Technologies utilisées

Couche	Technologie	Version	Rôle
Frontend	Flutter / Dart	3.x / 3.4+	Application mobile iOS & Android
Backend	Laravel	12.0	API REST sécurisée
Auth	Laravel Sanctum	4.0	Tokens Bearer
Paieement	Stripe	stripe-php 19.4	Recharge par carte
Blockchain	Algorand	SDK	Cryptomonnaie BKN
Reverse Proxy	Nginx	Nginx-1.28.2	Protection serveur
E-mail	Resend	resend-laravel 1.3	Notifications transactionnelles
Base de données	PostgreSQL	postgresql-18.1-2	Persistance des données
Versioning	GitHub	—	Versioning

2. Arborescence du projet

Le projet est organisé en trois dossiers principaux au sein du dépôt GitHub :

2.1 Structure générale

```
project/
├─ firstapp/           → Application Flutter (frontend principal)
├─ secondapp/          → Application Flutter secondaire (expérimentale)
└─ mon_api_flutter/    → API Laravel (backend)
```

2.2 Application Flutter (firstapp/)

```
firstapp/lib/
├─ main.dart           # Point d'entrée, init Stripe & auto-
login
├─ login_screen.dart   # Écran de connexion
├─ register_screen.dart # Inscription (avec PIN)
├─ main_navigation.dart # Navigation bas de page
├─ models/
│   └─ transaction.dart # Modèle de transaction
│   └─ transfer_response.dart # Réponse de virement
├─ screens/
│   └─ transfer_screen.dart # Écran principal
│   └─ profile_screen.dart # Profil utilisateur
│   └─ receive_money_screen.dart # QR code personnel
│   └─ scanner_screen.dart # Scanner QR
└─ services/
    └─ api_service.dart # Client HTTP (singleton)
    └─ stripe_service.dart # Service Stripe
```

2.3 Backend Laravel (mon_api_flutter/)

```
mon_api_flutter/app/
├─ Http/Controllers/
│   └─ AuthController.php # Auth, virements, historique
│   └─ StripeController.php # Paiements Stripe & webhooks
├─ Mail/
│   └─ LoginNotification.php
│   └─ WelcomeNotification.php
│   └─ TransferNotification.php
└─ Models/
    └─ User.php # Modèle utilisateur
    └─ Transfer.php # Virements P2P
    └─ Transaction.php # Transactions Stripe
```

2.4 Base de données — Migrations

Fichier de migration	Table créée	Description
----------------------	-------------	-------------

000...create_users_table	users	Utilisateurs (nom, email, mot de passe)
2026_02_26...balance	users (alter)	Ajout colonne solde (balance)
2026_03_17...pin	users (alter)	Ajout colonne transaction_pin
2026_02_26...transfers_table	transfers	Virements P2P (sender, receiver, amount)
2026_03_25...transactions_table	transactions	Transactions Stripe (type, référence)
000...personal_access_tokens	personal_access_tokens	Tokens Sanctum

3. Fonctionnalités développées

3.1 Authentification & Gestion de session

L'authentification est le socle de sécurité de l'application. Elle repose sur Laravel Sanctum pour la gestion des tokens Bearer.

3.1.1 Inscription

L'écran d'inscription (`register_screen.dart`) collecte les informations suivantes : nom complet, adresse e-mail, mot de passe (min. 8 caractères), confirmation du mot de passe, et un code PIN de transaction à 4 chiffres.

Côté backend, le contrôleur `AuthController::register()` valide les données, hash le mot de passe et le PIN avec `bcrypt` (`Hash::make`), crée l'utilisateur avec un solde de départ de 100,00 € dans le cadre du développement, envoie un e-mail de bienvenue via la classe `WelcomeNotification`, et retourne un token Sanctum.

À l'inscription, un bonus de bienvenue de 100,00 € est automatiquement crédité sur le compte, tel qu'annoncé dans l'e-mail de bienvenue envoyé au nouvel utilisateur.

3.1.2 Connexion & Auto-login

La connexion utilise l'e-mail et le mot de passe. En cas de succès, les anciens tokens de l'utilisateur sont révoqués et un nouveau token est généré. Ce mécanisme garantit qu'un seul appareil est connecté à la fois.

Le token est persisté localement via `SharedPreferences`. Au prochain démarrage de l'application, si un token valide est trouvé, l'utilisateur est redirigé directement vers l'écran principal sans devoir se reconnecter.

```
// main.dart - Auto-login
final prefs = await SharedPreferences.getInstance();
final String? token = prefs.getString('auth_token');
runApp(MyApp(isLoggedIn: token != null));
```

3.2 Portefeuille & Affichage du solde

L'écran principal (`transfer_screen.dart`) affiche en temps réel le solde de l'utilisateur, récupéré depuis l'endpoint `GET /api/user`. Le solde est rafraîchi à chaque action (virement, recharge) et peut être actualisé manuellement via un pull-to-refresh.

La salutation personnalisée utilise le nom stocké dans le singleton `ApiService`, récupéré lors de la connexion et conservé en mémoire pendant la session.

3.3 Virements sécurisés par PIN

La fonctionnalité de virement est le cœur métier de l'application. Elle permet d'envoyer de l'argent à un autre utilisateur via son adresse e-mail, avec validation par code PIN.

3.3.1 Flux complet du virement

Le virement se déroule en plusieurs étapes :

- L'utilisateur saisit l'e-mail du destinataire et le montant.
- Un modal bottom sheet s'affiche pour la saisie du code PIN (masqué).

- La requête POST /api/send-money est envoyée avec l'email, le montant et le PIN.
- Côté Laravel : vérification du PIN avec Hash::check(), vérification du solde, transaction atomique DB::transaction().
- En cas de succès : débit du sender, crédit du receiver, création de l'enregistrement Transfer, envoi de l'e-mail de confirmation.
- L'interface affiche un dialogue de succès avec le nouveau solde.

3.3.2 Sécurité du virement

Plusieurs garde-fous sont en place : vérification que l'expéditeur et le destinataire ne sont pas le même utilisateur, vérification préalable du solde disponible, et rollback automatique de la transaction en cas d'erreur.

3.4 Paiement par QR Code

Le paiement par QR code permet un échange rapide d'argent entre deux utilisateurs physiquement proches, sans avoir à saisir d'adresse e-mail.

3.4.1 Génération du QR Code

L'écran ReceiveMoneyScreen génère un QR code contenant un objet JSON encodé avec l'identifiant et le nom de l'utilisateur :

```
final String qrData = jsonEncode({"id": user['id'], "name":  
user['name']});
```

Le package qr_flutter est utilisé pour le rendu visuel du QR code. Ce QR est unique par utilisateur et peut être présenté à tout moment pour recevoir un paiement.

3.4.2 Scanner & Paiement

L'écran ScannerScreen utilise mobile_scanner pour accéder à la caméra de l'appareil. À la détection d'un QR valide, l'objet JSON est parsé pour extraire l'ID du destinataire. Un dialogue demande à l'utilisateur de saisir le montant à envoyer. La requête POST /api/qr-payment est ensuite envoyée sans nécessiter de PIN (paiement de proximité).

Note : Le QR code encode uniquement l'ID utilisateur et non son solde ou ses informations sensibles. La logique de débit/crédit est entièrement gérée côté serveur.

3.5 Recharge via Stripe

L'intégration Stripe permet aux utilisateurs de recharger leur portefeuille avec une carte bancaire réelle (mode test avec les cartes Stripe de test).

3.5.1 Flux de paiement Stripe

- L'utilisateur appuie sur 'Recharger mon compte' (montant fixé à 20€ dans la démo).
- Une requête POST /api/payment/intent crée un PaymentIntent côté Laravel.
- Le clientSecret retourné initialise la PaymentSheet Flutter.
- L'utilisateur complète son paiement dans l'interface Stripe native.
- Stripe envoie un événement payment_intent.succeeded au webhook /api/stripe/webhook.

- Le webhook vérifie la signature, crée la transaction et crédite le solde de l'utilisateur.

```
// StripeController.php - Création du PaymentIntent
$intent = PaymentIntent::create([
    'amount' => $request->amount * 100, // En centimes
    'currency' => 'eur',
    'metadata' => ['user_id' => Auth::id()],
]);
```

3.5.2 Protection anti-doublon

Le webhook implémente une protection anti-doublon : avant de créditer le solde, il vérifie en base de données que le `payment_intent.id` n'a pas déjà été traité. Cela évite les crédits en double en cas de réception multiple du même événement Stripe.

3.6 Notifications par e-mail

Le service d'e-mail utilise Resend comme fournisseur et des templates Blade pour le rendu HTML. Trois types de notifications sont implémentés :

Événement	Classe Mail	Contenu
Inscription	WelcomeNotification	Message de bienvenue + annonce du bonus 100€
Connexion	LoginNotification	Alerte sécurité : date, heure de connexion
Virement envoyé	TransferNotification	Confirmation montant + nom du destinataire

Les e-mails sont envoyés de manière asynchrone dans des blocs try/catch pour ne jamais bloquer le flux principal en cas d'erreur d'envoi.

3.7 Historique des transactions

L'endpoint `GET /api/transactions` retourne toutes les transactions de l'utilisateur (envoyées et reçues), triées par date décroissante. Chaque transaction inclut les noms du sender et du receiver via Eager Loading Eloquent (`with(['sender', 'receiver'])`).

Dans le frontend, les transactions sont différenciées visuellement : une flèche rouge vers le haut pour les envois, une flèche verte vers le bas pour les réceptions. Le signe + ou - est ajouté devant le montant en conséquence.

3.8 Blockchain Algorand — Cryptomonnaie BKN

L'application intègre la blockchain Algorand pour permettre l'échange de tokens BKN (une cryptomonnaie fictive créée pour le projet). L'SDK Algorand pour Dart/Flutter permet la création de wallets, la consultation de soldes et l'envoi de transactions sur la testnet Algorand.

Cette fonctionnalité illustre comment une application fintech peut intégrer des actifs numériques en complément d'un système bancaire classique.

3.9 Chatbot IA

Un chatbot d'assistance est intégré dans une branche dédiée. Il permet à l'utilisateur de poser des questions en langage naturel sur les fonctionnalités de l'application (comment effectuer un virement, comment lire son solde, etc.). Le chatbot est alimenté par une API d'IA générative et ses réponses sont contextualisées à l'univers bancaire de FirstApp.

4. Sécurisation de l'application

4.1 Authentification et tokens

L'application utilise Laravel Sanctum pour l'authentification par tokens Bearer. Chaque token est associé à un utilisateur précis et peut être révoqué à tout moment. Pour limiter les sessions actives à une seule, la méthode `$user->tokens()->delete()` est appelée avant la création du nouveau token lors de chaque connexion.

4.2 Protection des virements par PIN

Chaque virement nécessite la saisie du code PIN à 4 chiffres de l'utilisateur. Ce PIN est hashé en base de données avec `bcrypt` lors de l'inscription et vérifié côté serveur avec `Hash::check()` lors de chaque opération. Le PIN n'est jamais stocké en clair ni retourné dans les réponses API.

4.3 Hachage des mots de passe

Les mots de passe sont systématiquement hashés avec la fonction `bcrypt` de Laravel (`Hash::make()`) avant d'être persistés en base. Le facteur de coût par défaut (10 rounds) offre un bon équilibre entre sécurité et performance.

4.4 Validation CSRF et Webhook

Laravel active par défaut la protection CSRF sur toutes les routes. Pour le webhook Stripe, cette protection est désactivée de manière ciblée via la méthode `validateCsrfTokens(except: ['api/stripe/webhook'])`. En contrepartie, la signature du webhook est vérifiée avec `Webhook::constructEvent()` et le secret de webhook Stripe pour garantir l'authenticité des événements reçus.

4.5 Anti-doublon sur les transactions

Le webhook Stripe implémente une vérification d'idempotence en base de données. Avant tout crédit de solde, la présence du `payment_intent.id` dans la table `transactions` est vérifiée. Si l'entrée existe déjà, l'opération est ignorée. Ce mécanisme est essentiel pour les webhooks qui peuvent être renvoyés en cas d'échec de livraison.

4.6 Transactions atomiques

Les opérations de virement (débit du sender + crédit du receiver + création de l'enregistrement Transfer) sont encapsulées dans une transaction `DB::transaction()`. En cas d'erreur à n'importe quelle étape, toutes les modifications sont automatiquement annulées (rollback), garantissant la cohérence des données.

4.7 Séparation des routes publiques et protégées

Les routes API sont clairement séparées en deux groupes dans `routes/api.php` : les routes publiques (login, register, webhook) accessibles sans authentification, et les routes protégées par le middleware `auth:sanctum` qui nécessitent un token Bearer valide dans le header `Authorization`.

4.8 Persistance locale sécurisée

Le token d'authentification est stocké localement sur l'appareil via le package `shared_preferences`. Bien que ce stockage ne soit pas chiffré nativement, il est accessible uniquement par l'application elle-même. Pour une application en production, l'utilisation de `flutter_secure_storage` (qui utilise le Keychain iOS et le Keystore Android) serait recommandée.

5. Explication des API utilisées

5.1 Laravel Sanctum

Sanctum est la solution officielle de Laravel pour l'authentification des SPA et des applications mobiles via des tokens API. Contrairement à Passport (basé sur OAuth2), Sanctum propose une approche simplifiée et légère : les tokens sont des chaînes aléatoires stockées dans la table `personal_access_tokens`.

Choix : Sanctum a été choisi pour sa simplicité d'intégration et sa compatibilité native avec Flutter. Il ne nécessite pas de serveur OAuth et fonctionne avec un simple header Bearer.

5.2 Stripe

Stripe est l'une des plateformes de paiement les plus réputées au monde, notamment pour sa documentation exhaustive et ses SDK de qualité. Côté Laravel, le package `stripe/stripe-php` permet la création de `PaymentIntents` et la gestion des webhooks. Côté Flutter, `flutter_stripe` fournit la `PaymentSheet` native pour iOS et Android.

Flux : `PaymentIntent` → `PaymentSheet` → Confirmation → Webhook → Crédit du solde. Ce flux est conforme aux best practices PCI-DSS : les données de carte ne transitent jamais par les serveurs Laravel.

5.3 Resend

Resend est un service d'envoi d'e-mails transactionnels moderne, conçu pour les développeurs. Il offre une API simple, une haute délivrabilité, et un tableau de bord pour suivre les envois. L'intégration se fait via le package `resend/resend-laravel` qui s'intègre nativement avec le système de Mail de Laravel.

Choix : Resend a été préféré à Mailgun ou SendGrid pour sa facilité de configuration et son plan gratuit généreux, adapté à un projet universitaire.

5.4 Algorand

Algorand est une blockchain de troisième génération reconnue pour sa rapidité, ses faibles coûts de transaction et son empreinte carbone réduite. Le réseau testnet d'Algorand a été utilisé pour le développement et les tests, permettant de travailler avec de vrais smart contracts et des tokens sans engagement financier réel.

BKN : Le token BKN a été créé comme ASA (Algorand Standard Asset) sur la testnet Algorand. Les utilisateurs peuvent en acquérir et en échanger directement depuis l'application.

5.5 APIs Flutter (packages)

Package	Version	Usage
<code>http</code>	<code>^1.1.0</code>	Requêtes HTTP vers l'API Laravel
<code>flutter_stripe</code>	<code>^12.4.0</code>	Payment Sheet native Stripe
<code>shared_preferences</code>	<code>^2.5.4</code>	Persistance locale du token

qr_flutter	^4.1.0	Génération de QR codes
mobile_scanner	^7.2.0	Scan de QR codes via caméra

6. Difficultés rencontrées et solutions apportées

6.1 Problème CORS et accès réseau local

Problème : En développement, le frontend Flutter sur appareil physique ne pouvait pas accéder au backend Laravel sur localhost. L'adresse 127.0.0.1 est résolue sur l'appareil lui-même, non sur la machine de développement.

Solution : Utilisation de l'adresse IP locale du réseau (ex : 192.168.1.12) dans la configuration de l'ApiService Flutter. Configuration de CORS dans Laravel pour accepter les requêtes provenant de cette IP. Cette adresse est commentée dans le code pour faciliter le changement lors du déploiement.

6.2 Webhooks Stripe en développement local

Problème : Stripe a besoin d'une URL publique pour livrer les webhooks. En environnement de développement local, le serveur n'est pas accessible depuis Internet.

Solution : Utilisation de la Stripe CLI (stripe listen --forward-to localhost/api/stripe/webhook) qui crée un tunnel temporaire et forwarder les événements Stripe vers l'environnement local. La désactivation ciblée du CSRF pour la route webhook a également été nécessaire.

6.3 Compatibilité Stripe sur Android

Problème : La PaymentSheet de Stripe nécessite que l'Activity Android hérite de FlutterFragmentActivity et non de la FlutterActivity standard. L'application crashait au moment d'afficher le formulaire de paiement.

Solution : Modification de MainActivity.kt pour étendre FlutterFragmentActivity. Ajout des dépendances AppCompat et Material dans build.gradle.kts pour supporter le thème natif requis par Stripe.

6.4 Gestion des doublons de transactions

Problème : En cas de réseau instable, Stripe peut renvoyer le même événement webhook plusieurs fois, risquant un double crédit du solde.

Solution : Implémentation d'un check d'idempotence : avant tout crédit, on vérifie si le payment_intent.id existe déjà en base. La contrainte UNIQUE sur la colonne reference garantit l'unicité au niveau de la base de données.

6.5 Synchronisation du solde après virement

Problème : Après un virement réussi, le solde affiché dans l'interface n'était pas mis à jour immédiatement (cache de l'ApiService).

Solution : Appel explicite à _chargerHistorique() et _chargerSoldeInitial() après chaque opération modifiant le solde. Utilisation de Future.wait() pour les appels parallèles afin d'optimiser les temps de chargement.

6.6 Intégration d'un Chatbot et du BKN (crypto) :

Par manque de temps et soucis de débogage je n'ai pas eu le temps de mettre en place ces fonctionnalités afin qu'elles soient livrables et fonctionnelles.

7. Ce qu'il manque pour une publication sur les Stores

7.1 Prérequis communs iOS & Android

- Passage en production Stripe (clés live au lieu des clés test)
- Déploiement du backend Laravel sur un serveur de production avec HTTPS (SSL obligatoire pour Stripe)
- Remplacement des URL en dur (192.168.x.x) par l'URL de production dans le code Flutter
- Politique de confidentialité (Privacy Policy) hébergée publiquement
- Conditions générales d'utilisation (CGU)
- Tests unitaires et d'intégration complets
- Icône d'application définitive (1024x1024px pour iOS, densités multiples pour Android)
- Splash screen natif

7.2 App Store (iOS)

- Compte Apple Developer (99 \$/an)
- Certificat de distribution et profil de provisionnement
- Respect des App Store Review Guidelines (notamment section Finance)
- Privacy Manifest file (.xcprivacy) déclarant l'usage des APIs sensibles
- Conformité aux règles financières d'Apple (agrément Banque de France peut être requis)
- TestFlight pour les tests bêta avant soumission

7.3 Google Play Store (Android)

- Compte Google Play Developer (25 \$ unique)
- Keystore de production (clé de signature) — ne jamais perdre ce fichier
- Data Safety Form : déclarer les données collectées et leur usage
- Cibler API level 35 minimum (requis pour les nouvelles apps en 2025)
- Permissions caméra justifiées dans la Data Safety Form
- Audit de sécurité (App Bundle au lieu d'APK)

7.4 Conformité réglementaire

En tant qu'application bancaire, FirstApp serait soumise à une réglementation stricte en production :

- Directive DSP2 (Services de Paiement 2) pour les transactions européennes
- Agrément ACPR (Autorité de Contrôle Prudentiel et de Résolution) pour les services de paiement
- Procédures KYC (Know Your Customer) pour vérifier l'identité des utilisateurs

- Procédures AML (Anti-Money Laundering) pour détecter les transactions suspectes
- Conformité RGPD pour la gestion des données personnelles

Dans le cadre académique, ces contraintes réglementaires ne s'appliquent pas.
L'application est utilisée uniquement dans un environnement de test.

8. Architecture Frontend & Backend

8.1 Frontend Flutter

8.1.1 Pattern Singleton pour l'ApiService

L'ApiService est implémenté comme un Singleton en Dart pour garantir une instance unique accessible depuis tous les écrans. Cela évite la duplication du token d'authentification et permet un accès centralisé aux données de session (userId, userName).

```
static final ApiService _instance = ApiService._internal();  
factory ApiService() => _instance;  
ApiService._internal();
```

8.1.2 Gestion des états

La gestion d'état utilise le setState() natif de Flutter (approche simple adaptée à la taille du projet). Pour des projets plus grands, l'usage de Provider, Riverpod ou BLoC serait préconisé. Les écrans chargent leurs données via des FutureBuilder ou des appels explicites dans initState().

8.1.3 Navigation

La navigation principale utilise une BottomNavigationBar avec IndexedStack pour conserver l'état des pages lors des changements d'onglets. Les navigations secondaires (scanner, QR) utilisent Navigator.push() avec MaterialPageRoute. La déconnexion utilise Navigator.pushAndRemoveUntil() pour vider complètement la pile de navigation.

8.2 Backend Laravel

8.2.1 Architecture MVC

Le backend suit l'architecture MVC (Modèle-Vue-Contrôleur) de Laravel. Les contrôleurs (AuthController, StripeController) gèrent la logique métier, les modèles Eloquent (User, Transfer, Transaction) abstraient l'accès à la base de données, et les routes API définissent les endpoints publics et protégés.

8.2.2 Relations Eloquent

Le modèle Transfer définit deux relations BelongsTo vers User : une pour le sender_id et une pour le receiver_id. L'utilisation de with(['sender:id,name', 'receiver:id,name']) dans les requêtes permet de charger les noms des utilisateurs en une seule requête SQL optimisée (Eager Loading).

8.2.3 Système de notification par e-mail

Chaque classe Mail étend la classe Mailable de Laravel et définit un Envelope (sujet de l'e-mail) et un Content (template Blade). Les propriétés publiques de la classe sont automatiquement disponibles dans les templates Blade, permettant la personnalisation dynamique des e-mails.

9. Scalabilité & Perspectives d'évolution

9.1 Améliorations techniques immédiates

- Stockage sécurisé du token : migrer de `shared_preferences` vers `flutter_secure_storage` (Keychain/Keystore)
- URL de base configurable : utiliser des variables d'environnement Flutter (`--dart-define`) pour les URLs
- Gestion d'erreurs globale : intercepteur HTTP pour les erreurs 401 (redirection auto vers login)
- Loading states globaux : provider ou Riverpod pour éviter la duplication des indicateurs de chargement
- Tests : unit tests Dart pour les modèles et services, tests de widgets, tests d'intégration Laravel

9.2 Nouvelles fonctionnalités

- Notifications push via Firebase Cloud Messaging (FCM) pour les alertes de réception
- Authentification biométrique (Face ID, Touch ID) en remplacement du PIN
- Virements programmés et récurrents
- Système de plafonds de virement configurables
- Multi-devises avec taux de change en temps réel
- Détection de fraude par IA (analyse comportementale des transactions)

9.3 Infrastructure de production

- Containerisation avec Docker pour un déploiement reproductible
- Cache Redis pour les requêtes fréquentes (solde, profil utilisateur)
- Queue Laravel Horizon pour le traitement asynchrone des e-mails
- Load balancer et auto-scaling cloud (AWS, DigitalOcean, OVH)
- Monitoring applicatif (Sentry pour les erreurs, Grafana pour les métriques)
- Pipeline CI/CD GitHub Actions pour les tests et déploiements automatiques

10. Conclusion

Le projet FirstApp représente une réalisation ambitieuse dans le contexte d'un projet de L3 Informatique. Il couvre l'ensemble de la chaîne de développement d'une application fintech : de l'authentification sécurisée au paiement par carte bancaire en passant par les QR codes et la blockchain.

Sur le plan technique, le projet a permis aux étudiants de se familiariser avec des technologies industrielles réelles : Laravel pour le backend, Flutter pour le mobile, Stripe pour les paiements, et Algorand pour la blockchain. L'utilisation de GitHub a simulé les conditions de travail d'une équipe de développement professionnelle.

Les difficultés rencontrées (CORS, webhooks, compatibilité Stripe) ont été des opportunités d'apprentissage précieuses. Les solutions apportées sont conformes aux bonnes pratiques du secteur et documentées dans le code source.

L'application, dans sa forme actuelle, constitue une base solide pour une fintech caribéenne. Avec les ajustements réglementaires, de sécurité renforcée, et de tests nécessaires, FirstApp pourrait évoluer vers un produit publiable sur les stores mobiles.

FirstApp démontre qu'il est possible, dans le cadre universitaire, de concevoir et d'implémenter une application mobile bancaire fonctionnelle intégrant des technologies de pointe du secteur fintech.

Annexes

Annexe A — Routes API complètes

Méthode	Route	Auth	Description
POST	/api/register	Non	Inscription utilisateur
POST	/api/login	Non	Connexion & génération token
POST	/api/stripe/webhook	Non	Événements Stripe
GET	/api/test-connexion	Non	Test de connectivité
GET	/api/user	Oui	Infos utilisateur + solde
GET	/api/transactions	Oui	Historique transactions
POST	/api/send-money	Oui	Virement avec PIN
POST	/api/payment/intent	Oui	Créer un PaymentIntent
POST	/api/qr-payment	Oui	Paieement par QR code

Annexe B — Variables d'environnement (.env)

```
APP_NAME=FirstApp
APP_KEY=base64:...          # php artisan key:generate
DB_CONNECTION=pgsql         # ou mysql
STRIPE_SECRET=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
MAIL_MAILER=resend
RESEND_API_KEY=re_...
MAIL_FROM_ADDRESS=noreply@exemple.com
```

Annexe C — Dépendances Flutter (pubspec.yaml)

```
dependencies:
  flutter_stripe: ^12.4.0    # Paiement Stripe
  shared_preferences: ^2.5.4 # Persistance locale
  qr_flutter: ^4.1.0        # Génération QR
  mobile_scanner: ^7.2.0    # Scan QR caméra
  http: ^1.1.0              # Client HTTP
```